

软件设计与体系结构复习题（二）

按参考题型改编；含客观题解析与主观题要点

说明：本卷按参考题型编排，共含单项选择题 20 道、填空题 10 道、判断题 10 道、简答题 6 道、应用题 3 道。建议先独立作答，再查看文末参考答案。

一、单项选择题

1. 软件从需求、设计、开发、测试、维护直到退役的全过程称为（ ）。
 - A. 软件生命周期
 - B. 类生命周期
 - C. 进程调度
 - D. 编译周期
2. 下列属于面向对象基本机制的是（ ）。
 - A. 封装、继承、多态
 - B. 排序、查找、哈希
 - C. 编译、链接、装载
 - D. 输入、输出、打印
3. 在用例图中，与系统交互的外部用户或外部系统称为（ ）。
 - A. 类
 - B. 对象
 - C. 参与者
 - D. 过滤器
4. 下列属于用例之间关系的是（ ）。
 - A. 包含和扩展
 - B. 聚合和组合
 - C. 同步和异步
 - D. 上行和下行
5. “组件持续与环境交互，但交互次序难以预先确定”的系统更适合采用（ ）。
 - A. 批处理风格
 - B. 基于事件的隐式调用风格
 - C. 主程序-子程序风格
 - D. 两层 C/S 风格
6. 在控制驱动设计中，多个任务宏观同时推进、微观交替占用资源，属于（ ）。

- A. 顺序控制
 - B. 并发控制
 - C. 批量控制
 - D. 静态控制
7. 层次风格的主要优点不包括（ ）。
- A. 可测试性好
 - B. 可替换性好
 - C. 可扩展性好
 - D. 执行效率一定最高
8. 下列不属于数据流风格的是（ ）。
- A. 批处理风格
 - B. 管道-过滤器风格
 - C. 过程控制风格
 - D. 黑板风格
9. 管道-过滤器风格不太适合（ ）。
- A. 文本流水线处理
 - B. 编译器分阶段处理
 - C. 实时日志流处理
 - D. 强交互式界面控制
10. 基于事件的隐式调用风格的连接件通常是（ ）。
- A. 事件与处理过程的绑定机制
 - B. 完整磁盘文件
 - C. 类继承关系
 - D. 数据库主键
11. 黑板系统的组成部分通常不包括（ ）。
- A. 知识源
 - B. 黑板数据结构
 - C. 控制机制
 - D. 浏览器视图
12. 解释器风格通常以（ ）为核心执行机制。
- A. 解释引擎解释执行被解释程序
 - B. 浏览器直接访问数据库
 - C. 过滤器共享全局状态

D. 客户端保存全部业务逻辑

13. 反馈控制环风格的关键思想是（ ）。

- A. 输出信息反馈并用于修正控制行为
- B. 所有模块只能单向调用
- C. 所有数据必须一次性处理
- D. 所有节点地位完全对等

14. 三层 C/S 体系结构通常划分为（ ）。

- A. 表示层、功能层、数据层
- B. 浏览器层、按钮层、颜色层
- C. 传感层、打印层、音频层
- D. 父类层、子类层、接口层

15. 下列最符合 P2P 应用特征的是（ ）。

- A. 节点既可请求服务也可提供服务
- B. 所有请求必须由唯一主机处理
- C. 客户端只能显示静态页面
- D. 数据库只能本地部署

16. 中间件在分布式系统中的主要作用是（ ）。

- A. 屏蔽异构环境并提供公共服务
- B. 删除所有网络通信
- C. 替代用户界面
- D. 把所有节点变成浏览器

17. MVC 中，封装应用状态和业务规则的组件是（ ）。

- A. 模型
- B. 视图
- C. 控制器
- D. 管道

18. “正方形继承矩形，但重写设置宽高导致矩形使用者行为异常”常用于说明违反（ ）。

- A. 里氏代换原则
- B. 开闭原则
- C. 单一职责原则
- D. 接口隔离原则

19. 迪米特法则强调（ ）。

- A. 只与直接朋友通信

- B. 所有类都必须互相调用
- C. 接口越大越好
- D. 子类必须依赖父类实现细节

20. 下列关于依赖倒转原则的说法正确的是（ ）。

- A. 抽象不应依赖细节，细节应依赖抽象
- B. 高层模块必须直接 new 所有低层类
- C. 接口应随具体类频繁变化
- D. 只适用于数据库设计

二、填空题

1. 数据流风格主要包括批处理风格和_____风格。
2. 管道-过滤器风格中，_____负责数据处理，管道负责数据传递。
3. 仓库风格以_____为中心，组件通过共享数据源间接交互。
4. 黑板系统通常由知识源、黑板数据结构和_____组成。
5. 两层 C/S 中，系统通常由客户机和_____组成。
6. B/S 体系结构中，用户端通常通过_____访问系统。
7. MVC 中的 M、V、C 分别表示模型、视图和_____。
8. 健壮性是指软件在出现错误时仍能适当_____。
9. 可维护性强调软件系统能够较容易地被_____。
10. 里氏代换原则要求所有引用基类的地方都能透明地使用其_____对象。

三、判断题

1. 结构化方法把数据和处理过程紧密封装为对象。（ ）
2. 类图可以描述类、接口及其关联、依赖、泛化等静态关系。（ ）
3. 主程序-子程序风格把复杂任务分解为子任务，便于一定规模内开发和维护。（ ）
4. 仓库风格中，各组件通常通过中央数据区共享信息，而不是直接互相调用。（ ）
5. 事件隐式调用风格中，事件处理器的调用顺序总是由事件源硬编码决定。（ ）
6. 解释器风格灵活性较强，但通常执行效率低于直接编译运行。（ ）
7. 闭环反馈控制系统的输出不会影响输入端控制过程。（ ）
8. 三层 C/S 把业务逻辑从客户端和数据库中分离出来，有利于维护和扩展。（ ）
9. P2P 体系结构没有任何节点失效风险，因此一定比 C/S 可靠。（ ）
10. 依赖倒转原则通常需要通过接口或抽象类隔离具体实现变化。（ ）

四、简答题

1. 简述用例图和类图的作用差异。

答：

2. 简述主程序-子程序风格的优点和缺点。

答：

3. 说明基于事件的隐式调用风格的机制、优点和局限。

答：

4. 简述 P2P 体系结构风格的含义和特点。

答：

5. 简述 MVC 风格中模型、视图、控制器的职责及常见连接方式。

答：

6. 比较开闭原则、里氏代换原则和依赖倒转原则。

答：

五、应用题

1.

阅读下面 Java 事件处理代码，分析其软件体系结构风格中的组件和连接件。

```
import java.awt.*;
import java.awt.event.*;
class LoginFrame extends Frame implements ActionListener {
    Button loginBtn = new Button("登录");
    Label msg = new Label("未登录");
    void init(){
        setLayout(new FlowLayout());
        add(msg); add(loginBtn);
        loginBtn.addActionListener(this);
        setSize(260,100); setVisible(true);
    }
    public void actionPerformed(ActionEvent e){
        if(e.getSource()==loginBtn){ msg.setText("已提交登录"); }
    }
}
```

(1) 指出事件源、监听器、注册方法、事件对象和事件处理方法。

(2) 说明连接件如何体现隐式调用。

答：

2.

某在线考试系统支持考生浏览试卷、提交答案，教师组卷和阅卷，管理员维护题库。系统需要跨学校访问，客户端不能统一安装专用软件，同时要求业务规则集中管理。

(1) 请采用 B/S 并结合三层思想设计体系结构。

(2) 分析该设计相对两层 C/S 的优势。

答：

3.

某报表导出模块中，ReportService 直接依赖 PDFExporter、ExcelExporter、HtmlExporter，并通过 switch 判断导出格式。后续可能新增 Word、CSV 等格式。

(1) 判断该设计是否符合开闭原则，并说明原因。

(2) 按依赖倒转原则重构，画出关键类/接口关系。

答：

参考答案与解析

一、单项选择题

题号	答案	简析
1	A	软件生命周期覆盖软件从产生到停止使用的全过程。
2	A	面向对象以对象、类、封装、继承、多态等为核心。
3	C	Actor 表示系统外部的角色，可以是人或外部系统。
4	A	用例之间常见 include 和 extend 关系。
5	B	事件驱动适合响应用户输入、网络消息等不确定事件。
6	B	并发强调宏观同时、微观交替；并行强调物理同时执行。
7	D	层次穿越可能增加开销，效率不是其必然优势。
8	D	黑板属于数据共享/仓库类风格。
9	D	该风格数据流固定，不擅长复杂交互反馈。
10	A	事件发布后，系统根据注册关系自动调用处理过程。
11	D	黑板系统由知识源、黑板数据结构和控制机制组成。
12	A	解释器通过解释引擎逐步解释程序或规则。
13	A	闭环通过反馈比较和调节实现自适应。
14	A	三层 C/S 把界面、业务逻辑、数据访问分离。
15	A	P2P 节点具有客户机和服务器的相对双重角色。
16	A	中间件为分布式应用提供通信、事务、安全等公共支撑。
17	A	Model 保存核心数据和业务逻辑，View 负责显示，Controller 负责输入协调。
18	A	子类无法透明替代父类时违反 LSP。
19	A	LoD 又称最少知识原则，减少对象间不必要耦合。
20	A	DIP 通过稳定抽象隔离具体实现变化。

二、填空题

题号	答案	简析
1	管道-过滤器	两者都以数据传递驱动处理流程。
2	过滤器	过滤器是处理单元。
3	数据	中央数据区是组件通信媒介。
4	控制机制	控制机制决定知识源的调度。
5	服务器	客户机请求服务，服务器提供服务。
6	浏览器	Browser/Server 的客户端是浏览器。
7	控制器	Controller 负责输入与协调。
8	执行	可理解为容错和异常处理能力。
9	修改	维护包括改错、适应、完善等修改活动。
10	子类	子类应能替换父类而不破坏正确性。

三、判断题

题号	答案	简析
1	错	这是面向对象方法的特征。
2	对	类图是静态结构建模工具。
3	对	该风格适合顺序过程清晰的问题。
4	对	仓库风格以共享数据为核心。
5	错	调用顺序由注册和事件调度机制决定。

6	对	逐条解释带来额外开销。
7	错	这是开环；闭环会把输出反馈回来。
8	对	功能层集中处理业务规则。
9	错	P2P 可降低单点瓶颈，但并非没有失效风险。
10	对	抽象是高层与低层之间的稳定契约。

四、简答题参考要点

1. 简述用例图和类图的作用差异。

用例图用于需求分析，从外部参与者角度描述系统提供的功能、系统边界以及用例之间关系。

类图用于静态结构建模，描述类、接口、属性、方法以及关联、聚合、组合、依赖、泛化等关系。

简言之，用例图回答“系统要为谁做什么”，类图回答“系统由哪些类及关系构成”。

2. 简述主程序-子程序风格的优点和缺点。

优点：可以把复杂任务分解为较小子任务，控制结构清晰，便于一定规模内开发、测试和维护；是传统过程式程序中成功使用的结构。

缺点：程序规模很大时开发和测试困难；可重用性差、数据安全性差；数据结构改变会影响多个处理过程；难以适应 GUI 等事件驱动应用。

3. 说明基于事件的隐式调用风格的机制、优点和局限。

机制：事件源发布或触发事件，事件处理器预先注册对某类事件的监听，事件发生时系统自动调用处理过程，事件源无需知道响应者。

优点：组件松耦合、复用性好、易扩展，适合交互式 and 异步系统。

局限：触发者失去部分计算控制权，难确定处理顺序和影响范围，正确性推理和调试较复杂。

4. 简述 P2P 体系结构风格的含义和特点。

P2P 把客户机和服务器看成相对角色，系统中每个节点既可请求其他节点服务，也可向其他节点提供服务。

特点：节点对等、资源分布式存放、扩展性较强、可降低中心服务器瓶颈；但资源发现、安全、管理和一致性控制更复杂。

5. 简述 MVC 风格中模型、视图、控制器的职责及常见连接方式。

模型：封装应用数据、状态和业务规则。

视图：展示模型数据，提供用户可见界面。

控制器：接收用户输入，调用模型并选择或更新视图。

连接方式可以包含显式调用、隐式事件通知以及 HTTP 等机制。MVC 的核心价值是分离显示、输入控制和业务状态。

6. 比较开闭原则、里氏代换原则和依赖倒转原则。

开闭原则：软件实体应对扩展开放、对修改关闭，强调需求变化时通过扩展适应。

里氏代换原则：子类对象必须能够透明替换父类对象，强调继承关系的正确性。

依赖倒转原则：高层模块和低层模块都依赖抽象，抽象不依赖细节，强调通过接口隔离具体实现变化。

三者关系：良好的抽象与可替换子类有助于实现开闭。

五、应用题参考要点

1. 题目要点

(1) 事件源：loginBtn；监听器：this，即 LoginFrame 对象；注册方法：loginBtn.addActionListener(this)；事件对象：ActionEvent e；事件处理方法：actionPerformed(ActionEvent e)。

(2) 连接件体现为事件-过程绑定机制。注册时按钮事件源与监听器建立绑定；用户点击按钮后产生 ActionEvent，事件调度机制自动回调监听器的 actionPerformed 方法。按钮不需要直接知道具体业务处理逻辑，因此属于基于事件的隐式调用风格。

2. 题目要点

(1) 可设计为：

浏览器表示层 -> Web/业务功能层 -> 数据层

表示层：试卷页面、答题页面、教师阅卷页面、管理页面。

功能层：登录鉴权、组卷规则、考试计时、答案提交、自动判分、阅卷流程、权限控制。

数据层：用户、题库、试卷、答卷、成绩等数据存储。

(2) 相对两层 C/S，B/S 无需为不同学校大量安装专用客户端，升级维护集中在服务器端；业务规则集中在功能层，便于统一修改、权限控制和并发扩展；浏览器客户端更轻，跨平台访问更方便。

3. 题目要点

(1) 不符合开闭原则。新增导出格式时需要修改 ReportService 中的 switch 逻辑或创建逻辑，稳定业务类随需求变化而改变。

(2) 抽象 Exporter 接口，声明 export(report)等方法；PDFExporter、ExcelExporter、HtmlExporter、WordExporter、CsvExporter 实现 Exporter；ReportService 只依赖 Exporter 接口，具体实现由工厂、配置或依赖注入提供。

可画为：

ReportService --> Exporter(接口)

PDFExporter --|> Exporter

ExcelExporter --|> Exporter

HtmlExporter --|> Exporter

WordExporter --|> Exporter

CsvExporter --|> Exporter

新增格式时增加新的实现类即可，通常不修改 ReportService。